1

인공지능과 데이터 활용의 기초

- **데이터와 정보의 차이**

우리는 매일 수많은 숫자와 기록을 접합니다. 기온, 시험 점수, 경기 결과, 판매 수량, 클릭 횟수 등은 모두 데이터(Data) 입니다.

그러나 이러한 숫자들이 곧바로 의미를 가지는 것은 아닙니다. 데이터는 단지 사실이나 값의 기록일 뿐, 그 자체로는 무엇을 의미하는지 알 수 없습니다. 이 값이 어떤 상황에서 생성되었는지, 무엇을 나타내는지에 대한 설명이 없다면 단순한 숫자에 불과합니다.

1) 데이터(Data)

데이터는 가공되지 않은 원시 자료입니다. 해석되기 이전의 값, 즉 맥락이 없는 기록을 의미합니다. 예를 들어 다음과 같은 숫자가 있다고 가정해봅시다.

이 값들은 단순한 데이터입니다. 이 숫자가 무엇을 의미하는지 알 수 없습니다.

2) 정보(Information)

데이터는 가공되지 않은 원시 자료입니다. 데이터에 설명과 해석이 더해질 때, 우리는 그것을 정보라고 부릅니다.

앞의 숫자에 다음과 같은 설명을 추가해봅시다.

- **"이 값은 한 학생의 네 번의 수학 시험 점수이다."**

이제 이 숫자는 단순한 값이 아니라,그 학생의 학습 성취도를 나타내는 정보가 됩니다.

3) 데이터와 정보의 관계

데이터와 정보의 관계는 다음과 같이 정리할 수 있습니다.

- **데이터 + 맥락(해석) = 정보**

데이터는 "사실의 기록", 정보는 "의미를 가진 결과" 즉, 정보는 데이터보다 한 단계 높은 개념입니다. 데이터를 해석하고 분석하는 과정을 통해 만들어집니다.

4) 인공지능과 정보

인공지능은 데이터를 입력 받아 패턴을 분석하고, 그 결과를 정보로 만들어내는 시스템입니다.

- **경기 기록(데이터)을 분석하여 승률 예측(정보)을 생성**
- **영화 평점 데이터로 추천 목록이라는 정보를 생성**

따라서 데이터가 정확하지 않으면 잘못된 정보가 만들어질 수 있습니다.

데이터 구조와 파이썬 기초

인공지능과 데이터 분석을 수행하기 위해서는 데이터를 저장하고 처리하는 방법을 이해해야 합니다.

파이썬은 데이터를 다루기에 매우 적합한 언어이며, 특히 리스트(List), 딕셔너리(Dictionary), 그리고 pandas 라이브러리는 데이터 분석의 기본 도구로 사용됩니다.

리스트와 딕셔너리

1) 데이터 저장 방식 비교

① 리스트(List)

리스트는 순서가 있는 데이터 집합입니다.

여러 개의 값을 하나의 변수에 저장할 수 있습니다.

특징

- 순서(index)가 존재함
- 동일한 자료형을 여러 개 저장하기에 적합
- 반복문과 함께 자주 사용됨

예시

```
scores = [85, 92, 78, 90]
print(scores[0])
```

② 딕셔너리(Dictionary)

딕셔너리는 키(Key)와 값(Value) 의 쌍으로 구성됩니다.

이름이나 항목이 있는 데이터를 저장할 때 유용합니다.

특징

- 키를 이용해 값에 접근
- 항목별 데이터 저장에 적합
- 구조화된 데이터 표현 가능

예시

```
student = {
"이름": "홍길동",
"수학": 85,
"영어": 92
}
print(student["수학"])
```

③ 리스트와 딕셔너리 비교

2) 반복문과 조건문 활용

데이터 분석에서는 반복문과 조건문이 매우 중요합니다.

예시

```
scores = [85, 92, 78, 90]

for score in scores:
if score >= 90:
print("우수")
```

이처럼 반복문과 조건문을 이용하면 데이터를 자동으로 검사하고 분류할 수 있습니다.

CSV 파일 구조 이해

실제 데이터 분석에서는 수천, 수만 개의 데이터를 다룹니다. 이러한 데이터는 보통 CSV 파일 형태로 저장됩니다.

1) CSV란 무엇인가?

CSV는 Comma-Separated Values 의 약자로 쉼표로 구분된 데이터 파일입니다.

예시

```
이름,수학,영어
홍길동,85,92
이순신,78,88
```

2) 행(row)과 열(column)

- 행(row): 한 개체(예: 한 학생)의 정보
- 열(column): 같은 종류의 데이터 항목

예시

이름	수학	영어
홍길동	85	92
이순신	78	88

3) 정형 데이터의 특징

CSV 데이터는 정형 데이터(Structured Data) 입니다.

특징

- 일정한 형식을 가짐
- 열(column)이 고정되어 있음
- 표 형태로 정리 가능

이러한 구조 덕분에 컴퓨터가 쉽게 분석할 수 있습니다.

pandas 기초

대량의 데이터를 효율적으로 분석하기 위해 우리는 pandas 라이브러리를 사용합니다.

1) DataFrame 구조

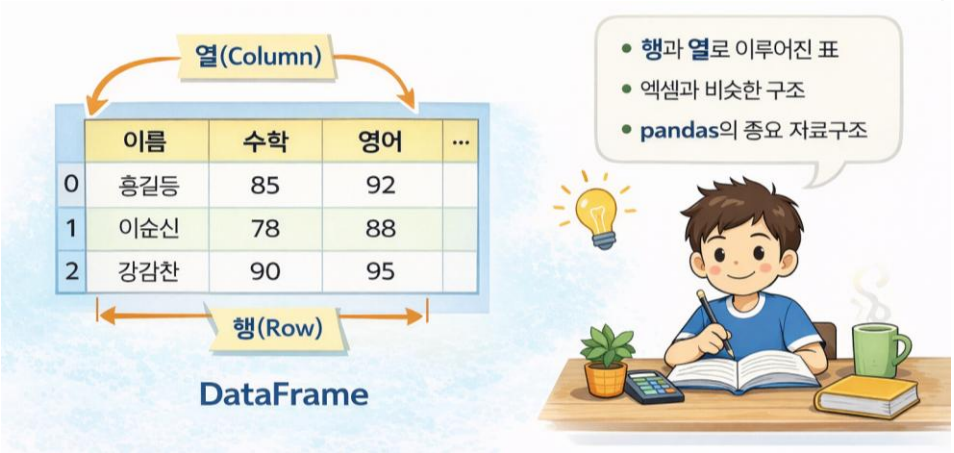
DataFrame은 행과 열로 구성된 표 형태의 자료 구조입니다. 엑셀과 매우 유사한 구조를 가집니다.

2) 데이터 불러오기

```
import pandas as pd

df = pd.read_csv("data.csv")
print(df.head())
```

- read_csv(): CSV 파일 읽기
- head(): 상위 5개 행 출력



3) 기본 함수

- **info( )**

데이터 구조와 자료형 확인.

```
df.info()
```

pandas 기초

- **describe( ) :** 기초 통계 요약

```
df.describe()
```

- **head() :** 상위 데이터 확인

```
df.head()
```

데이터 탐색과 기초 분석

데이터를 불러왔다면, 다음 단계는 데이터를 탐색(Exploration) 하는 것입니다.
데이터 탐색은 분석의 출발점입니다
어떤 값이 있는지, 이상한 값은 없는지 패턴은 무엇인지 확인하는 과정입니다.
이번 절에서는 다음 네 가지를 학습합니다.

- 데이터 정렬
- 조건 필터링
- 통계 요약
- 기초 시각화

데이터 정렬

데이터를 크기순으로 정렬하면 패턴을 더 쉽게 파악할 수 있습니다.

1) sort_values() 함수

```
import pandas as pd

df = pd.read_csv("data.csv")

df_sorted = df.sort_values("수학")
print(df_sorted)
```

- sort_values("열이름")
- 특정 열을 기준으로 정렬

2) 내림차순 정렬

```
df_sorted = df.sort_values("수학", ascending=False)
```

- ascending=False → 큰 값부터 정렬

3) 정렬의 의미

정렬을 통해 우리는 다음을 빠르게 확인할 수 있습니다.

- 가장 높은 점수는 누구인가?
- 가장 낮은 점수는 누구인가?
- 성적 분포는 어떻게 되는가?

정렬은 패턴을 발견하는 가장 기본적인 방법입니다.

조건 필터링

데이터 분석에서는 특정 조건을 만족하는 데이터만 추출하는 작업이 매우 중요합니다. 이를 필터링(Filter) 이라고 합니다.

1) 특정 조건 데이터 추출

예시: 수학 점수가 90점 이상인 학생 찾기

```
high_score = df[df["수학"] >= 90]
print(high_score)
```

구조 설명: `df[조건식]`

2) 여러 조건 사용하기

예시: 수학 80점 이상 AND 영어 90점 이상

```
result = df[(df["수학"] >= 80) & (df["영어"] >= 90)]
```

& : AND, | : OR

3) 필터링의 의미

필터링을 통해 우리는 우수 학생만 추출, 특정 구간 데이터 분석, 이상치 탐색 등을 수행할 수 있습니다.

이는 실제 빅데이터 분석에서 매우 자주 사용됩니다.

통계 요약

데이터의 전체적인 특성을 파악하기 위해 기초 통계를 계산합니다.

1) 평균 계산

```
print(df["수학"].mean())
```

2) 최댓값과 최솟값

```
print(df["수학"].max())
print(df["수학"].min())
```

3) describe() 함수 활용

```
df.describe()
```

출력 내용

- 평균(mean)
- 표준편차(std)
- 최소값(min)
- 최대값(max)
- 사분위수

3) 통계 요약의 의미

통계 요약을 통해 우리는 데이터의 중심 경향 파악, 데이터의 범위 확인, 값의 분포 이해를 할 수 있습니다.

이 과정은 인공지능 모델을 만들기 전에 반드시 수행해야 하는 단계입니다

기초 시각화

숫자만 보는 것보다 그래프로 표현하면 패턴을 더 쉽게 이해할 수 있습니다.

1) matplotlib 기본 사용

예시

```
import matplotlib.pyplot as plt

plt.bar(df["이름"], df["수학"])
plt.title("수학 점수 분포")
plt.xlabel("이름")
plt.ylabel("점수")
plt.show()
```

2) 2) 선 그래프 예시

예시

```
plt.plot(df["수학"])
plt.title("수학 점수 변화")
plt.show()
```

3) 시각화의 의미

그래프를 통해 우리는 점수 분포의 차이, 상승·하락 패턴, 이상치 여부를 직관적으로 파악할 수 있습니다.

시각화는 데이터 분석에서 결과를 설명하는 가장 강력한 도구입니다.

AI 도구를 활용한 코드 개선과 검증

앞 절까지 우리는 데이터를 정렬하고, 조건으로 필터링하며, 통계를 계산하고 시각화하는 방법을 실습했습니다.

이제는 한 단계 더 나아가, AI 도구(GPT) 를 활용해 코드를 더 빠르게 작성하고, 더 깔끔하게 개선하며, 오류를 찾아 수정하는 방법을 학습합니다.

중요한 점은 다음과 같습니다.

- **AI 도구는 "정답을 대신 만드는 도구"가 아니라 코드를 개선하고 검증하는 보조 도구**입니다.

코드 초안 작성 보조

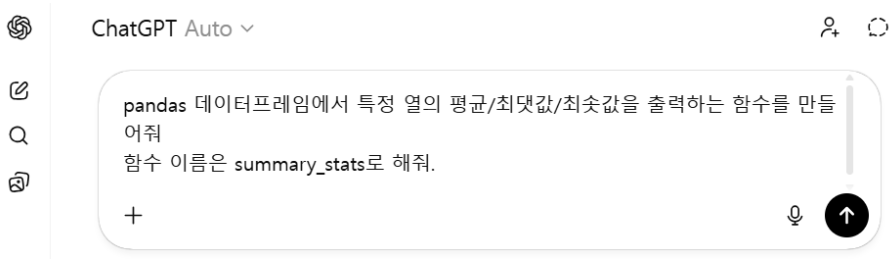
1) 반복되는 코드 자동 생성

데이터 분석에서는 비슷한 코드가 자주 반복됩니다. 예를 들어 평균, 최댓값, 최솟값을 각각 출력하는 코드는 다음처럼 반복될 수 있습니다.

```
print(df["수학"].mean())
print(df["수학"].max())
print(df["수학"].min())
```

이럴 때 GPT에게 "반복되는 코드를 함수로 정리" 하도록 요청할 수 있습니다.

[예시 프롬프트]



2) 생성된 초안 확인하기

AI가 작성해준 코드를 그대로 복사해서 사용하기보다 다음을 확인해야 합니다.

- 입력과 출력이 요구한 형식인지
- 변수, 열 이름이 실제 데이터와 일치하는지
- 불필요한 코드가 포함되어 있지 않은지

코드 설명 요청

1) 알고리즘 동작 원리 확인

AI 도구는 코드의 동작을 "설명해주는 역할" 에 매우 유용합니다. 특히 조건 필터링이나 정렬처럼 문법이 익숙하지 않을 때 도움을 받을 수 있습니다.

예시 프롬프트1

예시 프롬프트2

sort_values에서 ascending=False를 쓰면 어떤 결과가 나오고, 왜 그런지 설명해줘.

2) 코드 설명을 활용하는 방법

설명은 "이해를 돕는 참고 자료" 입니다. 설명을 읽은 뒤, 반드시 직접 다음을 해보세요.

- 코드를 실행해 결과 확인
- 값을 바꾸어 결과 변화 관찰
- 조건식을 바꿔 필터링 결과 비교

이 과정을 통해 문법이 **나의 지식**이 됩니다.

AI 도구를 활용한 코드 개선과 검증

코드 개선 요청

1) 시간복잡도 분석

알고리즘 문제를 풀 수 있는 수준이라면 코드가 동작하기만 하면 된다가 아니라 얼마나 효율적인가까지 고려해야 합니다.

예시 프롬프트

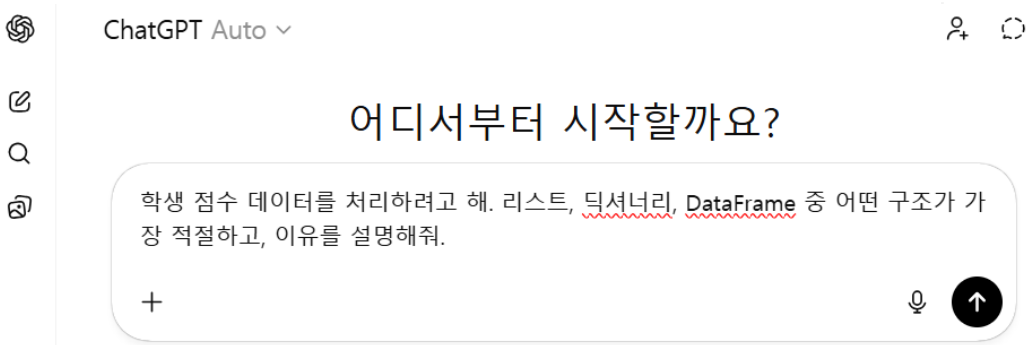
예를 들어, 반복문으로 평균을 직접 계산하는 코드가 있다면 반복문 기반 계산, pandas 내장 함수(mean) 사용 중 어느 것이 더 효율적이고 안정적인지 비교해볼 수 있습니다.

2) 더 효율적인 자료구조 제안

데이터를 저장할 때 자료구조 선택에 따라 코드가 단순해질 수 있습니다.

- 리스트: 순서 기반, 반복문에 유리
- 딕셔너리: 키 기반, 빠른 조회에 유리
- DataFrame: 표 형태, 분석, 필터링, 정렬에 유리

예시 프롬프트



검증 과정

AI 도구를 활용한 코딩에서 가장 중요한 단계는 검증입니다.

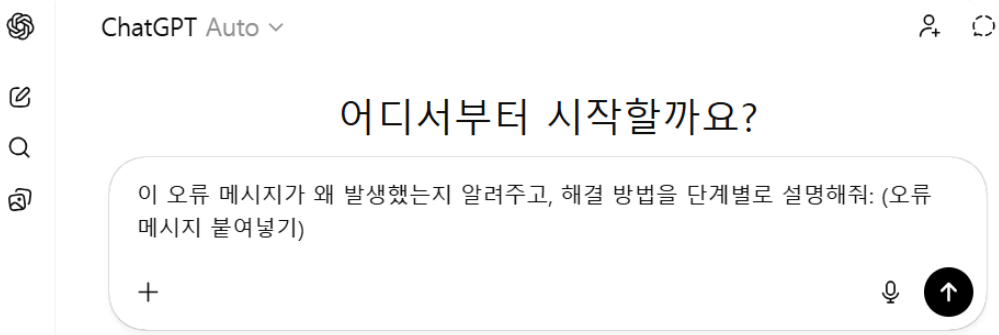
1) 직접 실행하기

가장 먼저 할 일은 코드가 실제로 실행되는지 확인하는 것입니다.

- 문법 오류(SyntaxError)
- 변수명 오타(NameError)
- 파일 경로 오류(FileNotFoundError)

오류가 나오면 오류 메시지를 그대로 AI에게 전달해도 됩니다.

예시 프롬프트



2) 엣지 케이스 테스트

AI가 만든 코드는 일반적인 데이터에서는 잘 동작하지만 특수한 상황에서는 오류가 나는 경우가 많습니다. 다음 경우를 엣지 케이스(Edge Case) 라고 합니다.

- 빈 데이터(행이 없는 경우)
- 결측치(NaN) 포함
- 문자열/숫자 혼합
- 특정 열이 존재하지 않는 경우

오류가 나오면 오류 메시지를 그대로 AI에게 전달해도 됩니다.

예시 프롬프트

데이터가 비어있을 때 평균을 계산하면 어떻게 될까?

특정 열 이름이 잘못되면 어떤 오류가 발생할까?

3) GPT 답변과 실제 결과 비교

AI 도구는 설명이나 결과를 제시할 때 정확하지 않을 수 있습니다. 따라서 다음을 습관화해야 합니다.

- AI가 말한 내용이 실제 결과와 일치하는지
- 수정 후 결과가 개선되었는지

실습 활동 및 과제 제출

이번 장에서는 데이터를 탐색하고 분석하는 방법을 학습했습니다. 또한 AI 도구(GPT)를 활용하여 코드를 개선하고 검증하는 방법도 살펴보았습니다.

이번 실습에서는 CSV 데이터를 직접 분석하고, 작성한 코드를 AI 도구를 이용해 개선하는 과정을 수행합니다.

실습 1. CSV 데이터 분석

다음 CSV 데이터를 사용하여 간단한 데이터 분석을 수행해봅시다.

- 예시 데이터 (data.csv)

```
이름,수학,영어
홍길동,85,92
이순신,78,88
강감찬,90,95
유관순,88,91
```

1) 데이터 불러오기

pandas 라이브러리를 이용하여 CSV 파일을 불러옵니다.

```
import pandas as pd

df = pd.read_csv("data.csv")
print(df)
```

2) 평균 계산

수학 점수의 평균을 계산해봅시다.

```
print(df["수학"].mean())
```

실습 활동 및 과제 제출

다음 질문에 답해봅시다.

- 수학 점수 평균은 얼마인가?
- 가장 높은 점수는 누구인가?

3) 조건별 데이터 출력

수학 점수가 90점 이상인 학생을 출력해봅시다.

```
high_score = df[df["수학"] >= 90]
print(high_score)
```

추가로 다음 조건을 작성해봅시다.

- 수학 85점 이상인 학생
- 영어 90점 이상인 학생

실습 2. GPT 활용 코드 개선

이번에는 작성한 코드를 AI 도구(GPT) 를 활용하여 개선해봅니다.

1) 작성한 코드 GPT에 전달

다음과 같이 GPT에게 질문해봅니다.

예시 질문

```
이 코드를 더 효율적으로 개선해줄 수 있을까?
또한 코드의 시간복잡도를 설명해줘.
```

```
이 코드를 더 읽기 쉽게 리팩토링해줘.
동일한 기능을 유지하면서 코드 구조를 개선해줘.
```

2) 개선 코드 비교

GPT가 제안한 코드와 기존 코드를 비교해봅시다.

비교 항목

비교 항목	기존 코드	GPT 개선 코드
코드 길이		
가독성		
실행 결과		

3) 시간복잡도 분석

GPT에게 다음 질문을 해봅니다.

```
이 코드의 시간복잡도는 무엇인가?
더 효율적인 방법이 있다면 제안해줘.
```

확인할 내용

- 반복문 사용 여부
- pandas 내장 함수 활용 여부
- 코드 실행 속도 차이